GitHub Actions → AWS ECR Using OIDC

A beginner-friendly DevOps guide: what OIDC is, what you configure in AWS, what you configure in GitHub Actions, and how the Docker image reaches ECR.

1. First understand the problem

Suppose a developer pushes code to GitHub. A GitHub Actions workflow builds a Docker image and needs to push that image to Amazon ECR. AWS must first know: “Who is this GitHub Actions workflow, and what is it allowed to do?”

```
Developer
   |
   | git push
   v
GitHub Repository
   |
   v
GitHub Actions
   |
   | authenticate to AWS
   v
AWS IAM
   |
   | allowed ECR actions
   v
Amazon ECR
   |
   v
Docker image stored
```

2. What is OIDC?

OIDC means OpenID Connect. In this setup, OIDC is an identity mechanism. It lets GitHub Actions prove to AWS that a workflow really came from GitHub and provides identity information such as the repository and workflow context.

> The easiest mental model is: **OIDC = identity verification**. **IAM Role = permissions.** OIDC does not itself give GitHub permission to push to ECR. The IAM Role does that.

```
GitHub Actions
      |
      | OIDC identity token
      v
AWS STS
      |
      | AssumeRole
      v
IAM Role
      |
      | ECR permissions
      v
Amazon ECR
```

3. Is the GitHub OIDC provider URL standard?

For GitHub.com Actions using AWS, the standard GitHub OIDC provider URL is:

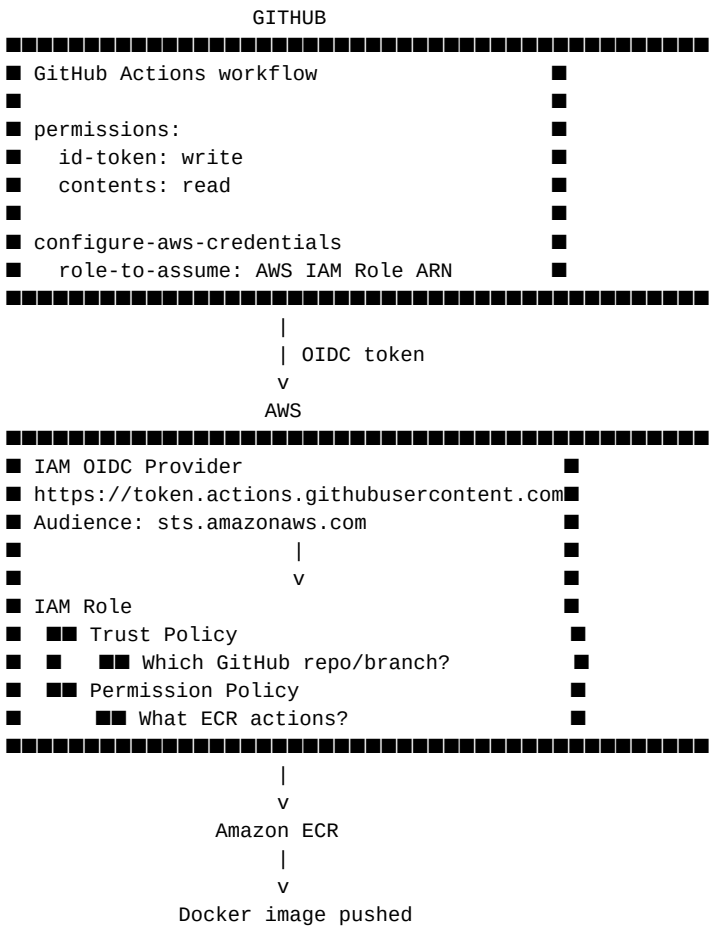
```
https://token.actions.githubusercontent.com
```

The standard audience used with the AWS credentials action is:

```
sts.amazonaws.com
```

These values are normally the same across GitHub repositories. What changes from project to project is mainly the IAM Role trust policy: it should restrict which GitHub repository, organization, and often branch/environment can assume the role.

4. The complete setup — the big picture



5. AWS side — Step 1: Add GitHub as an OIDC Identity Provider

In the AWS console, open **IAM** → **Identity providers** → **Add provider**. Select **OpenID Connect**.

Field	Value
Provider type	OpenID Connect
Provider URL	https://token.actions.githubusercontent.com
Audience	sts.amazonaws.com

Think of this step as telling AWS: **“GitHub is a trusted identity provider.”** It does not yet mean every GitHub repository can use your AWS account.

6. AWS side — Step 2: Create the IAM Role

Create a role that GitHub Actions will assume. The role is the AWS identity whose permissions will be used by the workflow.

The role has two very important parts:

A. Trust Policy — answers: “Who is allowed to assume this role?”
B. Permission Policy — answers: “What can this role do after it is assumed?”

```
IAM Role
  |
  +-- Trust Policy
  |     Who can assume me?
  |     -> GitHub OIDC
  |     -> specific repository/branch
  |
  +-- Permission Policy
        What can I do?
        -> ECR push permissions
```

7. AWS side — Step 3: Trust Policy

This is the security-critical part. Do not simply allow every identity from GitHub. Restrict the role to your intended GitHub repository and, where appropriate, a branch or environment.

Conceptual example:

```
Trusted identity provider:
    token.actions.githubusercontent.com

Allowed repository/branch:
    repo:YOUR_ORG/YOUR_REPO:ref:refs/heads/main

Meaning:
    Only the intended GitHub repository's main branch
    should be able to assume this role.
```

The exact JSON trust policy can vary depending on whether you restrict by branch, environment, or other GitHub OIDC claims. The important interview concept is: **the trust policy controls who may assume the role.**

8. AWS side — Step 4: Give the role ECR permissions

Attach a least-privilege IAM policy to the role. For pushing an image, the role needs the ECR actions required for authentication and image/layer upload. Common actions include:

```
ecr:GetAuthorizationToken
ecr:BatchCheckLayerAvailability
ecr:InitiateLayerUpload
ecr:UploadLayerPart
ecr:CompleteLayerUpload
ecr:PutImage
```

For production, restrict permissions as tightly as practical, including repository-level resource restrictions where supported. Avoid giving broad AdministratorAccess just to make the pipeline work.

9. GitHub Actions side — Step 1: Allow OIDC token requests

In the workflow YAML, give the workflow permission to request an OIDC token:

```
permissions:
  id-token: write
  contents: read
```

id-token: write is the key permission for OIDC. Without it, the workflow cannot request the GitHub OIDC token needed for this authentication flow.

10. GitHub Actions side — Step 2: Tell the workflow which IAM Role to assume

Use the official AWS credentials action and provide the ARN of the IAM Role created in AWS:

```
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v5
  with:
    role-to-assume: arn:aws:iam::123456789012:role/GitHubActions-ECR-Role
```

```
    aws-region: ap-south-1
```

The role ARN is not an AWS access key. GitHub uses OIDC to obtain temporary credentials by assuming this role.

11. Then authenticate Docker to ECR

Once AWS credentials are configured, the AWS CLI can obtain an ECR authorization token. Docker uses that token to authenticate to the ECR registry.

```
aws ecr get-login-password --region ap-south-1 |
docker login   --username AWS   --password-stdin   <account-id>.dkr.ecr.ap-south-1.amazonaws.com
```

After login, tag the image with the ECR repository URI and push it:

```
docker build -t myapp:latest .

docker tag myapp:latest   <account-id>.dkr.ecr.ap-south-1.amazonaws.com/myapp:latest

docker push   <account-id>.dkr.ecr.ap-south-1.amazonaws.com/myapp:latest
```

12. Full GitHub Actions flow

```
Developer
   |
   | git push
   v
GitHub Repository
   |
   v
GitHub Actions starts
   |
   | id-token: write
   v
GitHub creates OIDC identity token
   |
   v
AWS STS
   |
   | validates token + trust policy
   v
Assume IAM Role
   |
   | temporary AWS credentials
   v
AWS ECR permissions
   |
   v
ECR login
   |
   v
docker push
   |
   v
Amazon ECR
```

13. What goes where? — memorize this table

Item	Where?	Purpose
GitHub OIDC Provider URL	AWS IAM	AWS knows/trusts GitHub as an identity provider
Audience: sts.amazonaws.com	AWS IAM OIDC Provider	Identifies the intended AWS STS audience
IAM Role	AWS IAM	Identity GitHub assumes
Trust Policy	IAM Role	Controls who can assume the role
ECR Permission Policy	IAM Role	Controls what the role can do in ECR
id-token: write	GitHub Actions YAML	Allows workflow to request OIDC token
Role ARN	GitHub Actions YAML	Tells workflow which AWS role to assume
AWS region	GitHub Actions YAML	Selects AWS region
Docker login/push	GitHub Actions YAML	Authenticates Docker and pushes image

14. What is standard and what changes per project?

Usually standard for GitHub.com → AWS: the GitHub OIDC provider URL and the normal STS audience.

Project-specific: the IAM Role, trust policy restrictions, repository/branch/environment conditions, AWS account, ECR repository, region, and ECR permissions.

```
Project A:
GitHub repo A -> Role A -> ECR repo A

Project B:
GitHub repo B -> Role B -> ECR repo B

Both projects can use the same GitHub OIDC provider,
but their IAM roles should have different/restricted trust
and permission policies.
```

15. Common confusion cleared up

“Does OIDC give ECR permission?”
No. OIDC proves identity. IAM permissions determine what the assumed role can do.

“Do I put AWS access keys in GitHub Secrets?”
With OIDC, you normally do not need long-lived AWS access keys for this authentication flow.

“Is the OIDC provider different for every repository?”
No. The GitHub OIDC provider is shared; the IAM trust policy should restrict the repository/branch/environment.

“Is the IAM Role the same as the OIDC provider?”
No. The OIDC provider establishes trust in GitHub's identity tokens. The IAM Role is the identity GitHub assumes and whose permissions it receives.

“Does Docker directly understand OIDC?”
No. GitHub uses OIDC to obtain AWS temporary credentials; AWS/ECR authentication then provides Docker with an ECR authorization token.

16. Interview answer

If an interviewer asks, “How does GitHub Actions authenticate with AWS to push an image to ECR?”, a strong answer is:

> “I use GitHub OIDC federation instead of long-lived AWS access keys. In AWS, I configure GitHub's OIDC provider and create an IAM Role with a trust policy restricted to the required GitHub repository/branch. The role has least-privilege ECR push permissions. In GitHub Actions, I grant id-token: write permission and use the AWS credentials action to assume that role. AWS STS returns temporary credentials, and the workflow uses those credentials to authenticate to ECR and push the Docker image.”

17. Final memory trick

```
OIDC
  = Who are you?

IAM Role
  = What identity will you use in AWS?

Trust Policy
  = Who is allowed to use this role?

Permission Policy
  = What can this role do?

GitHub Actions YAML
  = Ask for OIDC + assume the role

ECR
  = Where the Docker image is stored
```

> If you remember only one line: **GitHub proves its identity using OIDC → AWS STS lets it assume an IAM Role → the Role has ECR permissions → Docker pushes the image to ECR.**

Reference: GitHub Actions documentation on OIDC with AWS and AWS IAM/ECR documentation. Values and implementation details can evolve, so verify against current official documentation when implementing in production.